



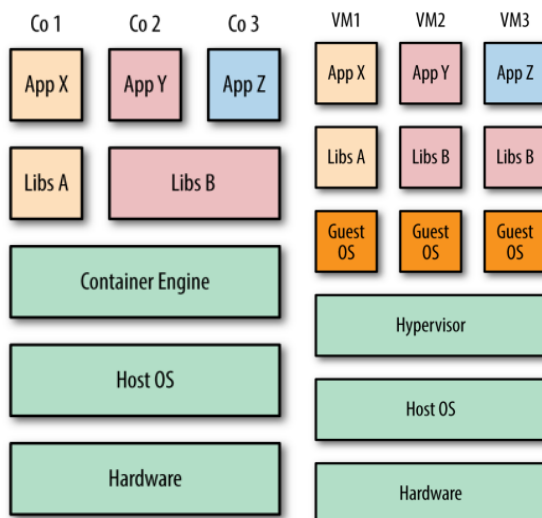
Docker

David Darras

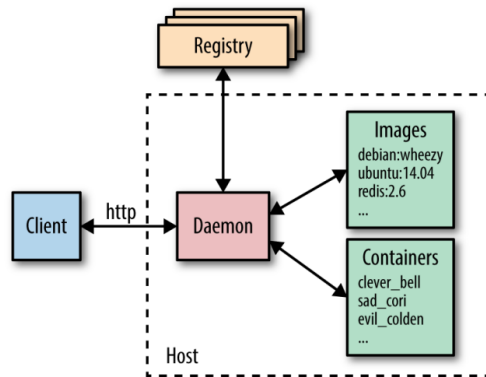
Date de création : 23/09/2025
Dernière modification : 27/09/2025

1 Préambule

- Une **image** est un modèle immuable en lecture seule, constitué d'un système de fichiers et de métadonnées telles qu'une application, ses bibliothèques, ses dépendances, des variables d'environnement, une configuration, etc. Elle peut être vue comme un snapshot d'un environnement d'exécution, construit par superposition de couches.
- Un **conteneur** est une instance en cours d'exécution d'une image. Lorsqu'il est créé, une couche en lecture-écriture est ajoutée au-dessus de l'image, ce qui permet d'appliquer des modifications temporaires (perdues à l'arrêt du conteneur, sauf si elles sont stockées dans un volume). On peut le comparer à la distinction entre un exécutable (l'image) et un processus (le conteneur).
- **Attention :** Contrairement aux machines virtuelles (VM), qui dupliquent l'OS et ses bibliothèques pour chaque application, les conteneurs mutualisent le noyau et certaines ressources de l'hôte. Dès lors, ils sont plus efficaces (lancement et arrêt en quelques secondes), portables, et plus légers (pas de duplication du noyau). Alors qu'une VM vise à simuler un environnement complet, un conteneur cherche à rendre les applications portables et autonomes tout en optimisant l'usage des ressources, avec une isolation au niveau des processus.



- Pour résumer, les conteneurs Docker ont apporté une solution élégante au problème classique du "ça marche sur ma machine, mais pas en production". Avant Docker, le déploiement d'une application dépendait fortement de l'environnement cible : système d'exploitation, versions des bibliothèques, configurations locales, etc. Docker simplifie le transport et l'exécution des applications logicielles en les empaquetant avec toutes leurs dépendances dans une unité portable et cohérente. Ainsi, les développeurs peuvent se concentrer sur la construction de l'application et l'acheminer à travers les différentes phases (développement, test, production) sans se soucier des différences d'environnement. Que l'application tourne sur un laptop, dans un cluster ou dans le cloud, le comportement reste identique.
- Le **daemon Docker** (`dockerd`) est le processus principal qui gère Docker. Il est responsable de la création, de l'exécution et de la surveillance des conteneurs, ainsi que de la construction, du téléchargement et du stockage des images en local.
- Le **client Docker** est l'interface en ligne de commande (`docker`) qui permet à l'utilisateur d'interagir avec le daemon Docker. Il traduit chaque commande en requêtes HTTP envoyées au daemon, généralement via un socket Unix (en local), ou via TCP (pour un daemon distant). Par exemple, lorsqu'on exécute `docker ps`, le client envoie une requête au daemon pour obtenir la liste des conteneurs actifs.
- Les **registres Docker** sont des serveurs distants qui stockent et distribuent les images. Ils permettent de partager facilement des images entre développeurs. Le registre par défaut est le **Docker Hub**, un service cloud qui héberge un grand nombre d'images publiques (par exemple `ubuntu`, `nginx`, `postgres`).



- L'Union File System (UFS), nommé **OverlayFS** sur Docker, est le mécanisme de système de fichiers superposé utilisé pour stocker les images en couches. Ce fonctionnement permet de partager efficacement les couches communes entre plusieurs images ou conteneurs, d'économiser de l'espace disque et d'accélérer la construction d'images grâce à un système de cache.

- Docker propose plusieurs outils complémentaires au moteur principal :

- **Swarm** : solution de clustering Docker permettant de répartir la charge, d'assurer la redondance et d'améliorer la disponibilité des services. Pour des besoins d'orchestration plus avancés ou dans des environnements hétérogènes, **Kubernetes** est aujourd'hui l'outil privilégié pour déployer, gérer et faire évoluer des applications multi-conteneurs sur des clusters.
- **Compose** : outil pour définir et gérer facilement des applications multi-conteneurs, particulièrement utile en développement et en test.
- **Machine** : permet d'installer et configurer rapidement des hôtes Docker (moins utilisé aujourd'hui, remplacé en partie par des solutions cloud).
- **Kitematic** : interface graphique pour gérer les conteneurs (outil historique, remplacé en partie par **Docker Desktop**).
- **Docker Trusted Registry** : solution locale pour stocker et sécuriser les images dans un registre privé. Pour des alternatives modernes open-source, on peut utiliser **Harbor**, qui offre un registre local ou cloud, avec gestion des rôles et scans de vulnérabilités.

2 Commandes docker à retenir

```

# -----
# Configuration de l'utilisateur Docker
# -----
# Docker est un binaire privilégié (nécessite sudo)
# Pour éviter d'utiliser "sudo" à chaque fois :
sudo usermod -aG docker $USER
# Appliquer les changements : relancer la session ou redémarrer le
↳ service
sudo service docker restart

```

```

# Vérifier que Docker est bien installé
docker version # Versions du client et du serveur Docker

# Test rapide pour vérifier Docker
docker run debian echo "Hello World"

# -----
# Lancer un conteneur interactif
# -----
# Options importantes :
# -it : interactif + TTY (permet de lancer un shell)
# -d : détaché, conteneur en arrière-plan
# -h : définir le nom d'hôte UNIX du conteneur
# -p : rediriger un port hôte vers le conteneur
# --name : nom du conteneur

# Exemple n°1 : ouvrir un shell dans le conteneur
docker run -it -h monubuntu --name TestUbuntu ubuntu:22.04 bash
# ubuntu est l'image de base à utiliser et :22.04 est la version
↳ souhaitée
# bash est l'application qu'on souhaite exécuter
# Docker préconise d'utiliser un seul processus par conteneur !

# Exemple n°2 : serveur Nginx exposé sur le port 8080 de l'hôte
docker run -d -p 8080:80 nginx
# -p 8080:80 : redirection du port hôte 8080 vers le port 80 du
↳ conteneur

# Monter un volume pour persister les données
docker run -it -v /home/user/data:/data --name monconteneur
↳ ubuntu:22.04 bash
# Les fichiers dans /data sur l'hôte sont persistants même après
↳ l'arrêt du conteneur

# -----
# Gestion des conteneurs
# -----
docker ps # Liste des conteneurs en cours (-a pour tous)
docker top monconteneur # Liste des processus en cours dans le
↳ conteneur
docker logs monconteneur # Affiche les logs
docker inspect monconteneur # Informations détaillées sur un
↳ conteneur
docker diff monconteneur # Modifications de fichiers dans le
↳ conteneur
docker pause monconteneur # Suspendre tous les processus
docker unpause monconteneur # Reprendre
docker start monconteneur # Relancer le conteneur
docker rm monconteneur # Supprimer le conteneur
docker rm -v $(docker ps -aq -f status=exited) # Supprimer tous les
↳ conteneurs arrêtés

# -----
# Publier ou récupérer des images
# -----
docker pull ubuntu:22.04 # Télécharger une image
docker push moncompte/monimage:maversion # Envoyer une image locale
↳ vers Docker Hub

# Trois espaces de noms :
# 1) utilisateur Docker Hub : darras/cowsay ou ogruber/java
# 2) root pour les images officielles : ubuntu, debian
# 3) images hébergées ailleurs : 197.1.72.4:5000/wordpress

# -----
# Construire une image Docker à partir d'un Dockerfile
# -----
docker build -t monrepo/monimage .
# -t : tag de l'image
# '.' : contexte de construction (fichiers disponibles pour
↳ COPY/ADD)

# Inspecter les couches d'une image
docker history monrepo/monimage

# -----
# Réseaux et liens entre conteneurs
# -----
# Chaque conteneur est isolé par défaut (réseau, processus,
↳ fichiers)
# --link permet de relier un conteneur à un autre pour qu'ils
↳ puissent communiquer

docker run --name mypostgres -e POSTGRES_PASSWORD=secret -d
↳ postgres:15
docker run -it --name myspringapp --link mypostgres:db -p 8080:8080
↳ springboot-app
# myspringapp peut accéder à la base de données via le nom "db"
# Docker met automatiquement une entrée dans /etc/hosts pour le
↳ conteneur lié

```

3 Création d'une image Docker

Le **Dockerfile** est un fichier texte contenant un ensemble d'étapes pour automatiser la création d'une image Docker. Chaque instruction crée une nouvelle couche d'image et peut être exécutée au moment de la construction (RUN) ou au démarrage du conteneur (CMD, ENTRYPOINT). Les paramètres peuvent être passés comme une commande shell : `CMD echo "Hello"` ou au format exec JSON : `CMD ["echo", "Hello"]`.

Structure d'un Dockerfile

```
# -----
# Image de base (TOUJOURS AU DEBUT)
# -----
FROM debian:latest # ou ubuntu:22.04, alpine (~5Mo), scratch
# Tag (version utilisée) : latest, wheezy, etc.

# -----
# Métadonnées : Créateur du conteneur
# -----
MAINTAINER David Darras <david.darras@gmail.com>

# -----
# Variables d'environnement
# -----
ENV JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64

# -----
# Copier des fichiers du contexte de construction vers l'image
# -----
COPY ./app /app
# Attention : le contexte de construction est le répertoire où est
↳ lancé docker build

# -----
# Instructions exécutées à la construction
# -----
RUN apt-get update && apt-get install -y curl
# Exécuté au build, le résultat est enregistré dans la couche
↳ d'image

# -----
# Script exécuté au démarrage du conteneur
# -----
ENTRYPOINT ["/app/start.sh"]

# -----
# Commande par défaut
# -----
CMD ["echo", "Hello World"]
# CMD peut être remplacé lors du run : docker run <image> <commande>

# -----
# Configuration réseau et volumes
# -----
EXPOSE 8080 # ports exposés
VOLUME /data # volume pour persister les données (TOUJOURS EN FIN)
```

Bonnes pratiques et astuces

- Utiliser un `.dockerignore` pour exclure les fichiers inutiles du contexte.

- Placer les instructions qui changent rarement en haut pour optimiser le cache (ex : `RUN apt-get update`).
- Toujours rendre exécutable les scripts : `chmod +x entrypoint.sh`
- Faire un `docker pull <image>` avant le build pour utiliser les dernières versions sécurisées.

Exemple concret : image "cowsay"

```
# Dockerfile
FROM debian:wheezy
RUN apt-get update && apt-get install -y cowsay fortune
COPY entrypoint.sh /
RUN chmod +x /entrypoint.sh
ENTRYPOINT ["/entrypoint.sh"]
```

```
# entrypoint.sh
#!/bin/bash
if [ $# -eq 0 ]; then
    /usr/games/fortune | /usr/games/cowsay
else
    /usr/games/cowsay "$@"
fi
```

```
# Construire et lancer l'image
docker build -t test/cowsay_img .
docker run test/cowsay_img "Hello Moo"
```

```
< Hello Moo >
      _
     (oo)\_____
    (__)\       )\/\
       ||----w |
       ||     ||
```

4 Références

